



Certificación de Python

Informe del Proyecto:

Sistema de Gestión de Inventarios para una Tienda de Abarrotes

Alumno: Julio Cesar Pantoja Barroso

**Estudiante de Ingeniería Biotecnológica del Instituto Politécnico
Nacional**

**Unidad Profesional Interdisciplinaria de Ingeniería Campus
Guanajuato**

Contenido

1. Introducción.....	3
2. Planteamiento del problema	3
2.1 Descripción de la problemática	3
2.2 Justificación e impacto potencial.....	3
2.3 Identificación de stakeholders	4
2.4 Objetivos	4
3. Base de datos de ejemplo, análisis exploratorio y limpieza	5
3.1 Descripción de la base de datos.....	5
3.2 Análisis exploratorio de datos (EDA)	5
3.3 Limpieza y preprocesamiento de datos	6
4. Evaluación del programa: funciones y técnicas utilizadas	7
4.1 Programación orientada a objetos (POO).....	7
4.2 Estructuras de datos avanzadas	8
4.3 Manipulación de datos con pandas y persistencia con JSON.....	8
4.4 Visualización con matplotlib	8
5. Complementación: control de datos y persistencia	9
5.1 Control de datos: duplicados y bajo stock	9
5.2 Persistencia de datos.....	9
5.3 Análisis estadístico visual.....	9
6. Instrucciones de uso paso a paso	11
Paso 1. Instalar las dependencias.....	11
Paso 2. Ejecutar el programa	11
Paso 3. Mostrar el inventario.....	11
Paso 4. Agregar un producto	12
Paso 5. Reabastecer y registrar ventas.....	12
Paso 6. Consultar bajo stock.....	13
Paso 7. Cargar la base de muestra y generar gráficas.....	13
7. Conclusión.....	14
8. Referencias	15

1. Introducción

El presente informe documenta el diseño, desarrollo y evaluación de un sistema de gestión de inventarios orientado a tiendas de abarrotes de pequeña escala, construido íntegramente en lenguaje Python. El proyecto integra los conocimientos abordados durante la certificación: fundamentos del lenguaje, programación orientada a objetos, estructuras de datos avanzadas, manipulación y análisis de datos, así como buenas prácticas de codificación y control de versiones. La elección de Python se justifica por su legibilidad, su amplia adopción y su ecosistema de bibliotecas para el análisis de datos, características que lo posicionan como uno de los lenguajes de mayor crecimiento en la actualidad (Van Rossum & Drake, 2009).

El documento se organiza en cinco bloques: el planteamiento del problema con su justificación basada en datos oficiales; la descripción de una base de datos de ejemplo junto con su análisis exploratorio y limpieza; la evaluación de las técnicas y funciones de programación utilizadas; la complementación referente al control de datos y la persistencia; y, finalmente, las instrucciones de uso y la conclusión. Cada apartado busca demostrar tanto el dominio técnico como la reflexión crítica sobre la utilidad del sistema en un contexto real (McKinney, 2010).

2. Planteamiento del problema

2.1 Descripción de la problemática

Las micro y pequeñas empresas constituyen el tejido productivo predominante de México. De acuerdo con los Censos Económicos 2019, el 99.8 % de los establecimientos del país pertenece a la categoría de micro, pequeñas y medianas empresas (mipymes), unidades que se caracterizan por una elevada movilidad en cuanto a aperturas y cierres (INEGI, 2024a). Dentro de este universo, las tiendas de abarrotes son uno de los formatos más extendidos y, a la vez, uno de los que opera con menores recursos administrativos y tecnológicos (INEGI, 2024a).

Un problema recurrente en estos negocios es la ausencia de un control sistemático del inventario. La gestión de existencias suele realizarse de memoria o mediante anotaciones manuales, lo que genera errores de captura, productos duplicados, desabastos no detectados a tiempo y decisiones de compra basadas en la intuición y no en datos. Esta carencia de herramientas de administración se ha identificado como una de las causas estructurales que limitan el crecimiento y la supervivencia de las pymes mexicanas (Ramírez Castillejo & Hernández Pérez, 2023).

La situación adquiere mayor relevancia al considerar la alta mortalidad empresarial del país. El Estudio sobre la Demografía de los Negocios 2023 estimó que, entre mayo de 2019 y mayo de 2023, nacieron 1.7 millones de establecimientos mipymes y murieron 1.4 millones; expresado como tasa mensual, por cada 10 mil establecimientos nacieron 61 y murieron 71 (INEGI, 2024b). Un control de inventario deficiente contribuye a este fenómeno al inmovilizar capital en mercancía sin rotación y al provocar pérdidas por caducidad o faltantes (INEGI, 2024b).

2.2 Justificación e impacto potencial

La justificación de un sistema de gestión de inventarios para tiendas de abarrotes se sustenta en evidencia sobre la relación entre el control de existencias y el desempeño del negocio. La

literatura especializada señala que una gestión adecuada de inventarios permite establecer políticas, métodos y procedimientos que reducen costos y mejoran la disponibilidad de productos, incidiendo directamente en la rentabilidad de las pymes (Romero-Agila et al., 2021). En el caso particular de las microempresas comerciales, el control de inventarios constituye una herramienta de información clave para la toma de decisiones gerenciales (Pin Figueroa et al., 2019).

El impacto potencial de implementar una herramienta sencilla y accesible es significativo. Al digitalizar el registro de productos, el comerciante puede detectar oportunamente los artículos con bajo stock, evitar la duplicación de registros y conocer el valor monetario inmovilizado en su inventario, lo que favorece decisiones de compra más informadas y una mejor planeación financiera (Romero-Agila et al., 2021). Asimismo, la adopción de tecnologías digitales accesibles se ha asociado con mayores probabilidades de supervivencia de las mipymes frente a entornos cambiantes (INEGI, 2024a).

2.3 Identificación de stakeholders

El desarrollo de este sistema involucra a diversas partes interesadas (stakeholders) cuyo análisis permite dimensionar su alcance. A continuación se identifican los actores clave y la forma en que el proyecto los impacta (Romero-Agila et al., 2021).

Stakeholder	Impacto
Propietario de la tienda	Principal beneficiario: obtiene control del inventario, reduce pérdidas y toma mejores decisiones de compra.
Empleados o dependientes	Usuarios operativos del sistema; agilizan el registro de ventas y reabastecimientos con menos errores.
Clientes de la tienda	Beneficiarios indirectos: mayor disponibilidad de productos y menos desabastos.
Proveedores	Se benefician de pedidos más precisos y oportunos basados en niveles reales de stock.
Desarrollador	Aplica y demuestra competencias técnicas de Python, POO y análisis de datos.

2.4 Objetivos

Objetivo general:

Desarrollar un sistema de gestión de inventarios en Python que permita a una tienda de abarrotes registrar, controlar y dar seguimiento a sus productos de forma sencilla, confiable y persistente, aplicando programación orientada a objetos y análisis de datos.

Objetivos específicos:

- Implementar las operaciones básicas de gestión de productos (alta, reabastecimiento, venta y eliminación) mediante una arquitectura orientada a objetos que garantice la integridad de los datos.

- Incorporar mecanismos de control de datos que eviten productos duplicados y alerten sobre niveles de bajo stock, empleando estructuras de datos avanzadas como diccionarios y conjuntos.
- Asegurar la persistencia de la información mediante archivos JSON y demostrar buenas prácticas de análisis y limpieza de datos con la biblioteca pandas.

3. Base de datos de ejemplo, análisis exploratorio y limpieza

3.1 Descripción de la base de datos

Para ejemplificar el funcionamiento del sistema se construyó una base de datos de muestra (archivo inventario_muestra.csv) que simula el registro manual de 24 productos típicos de una tienda de abarrotes mexicana, con los campos nombre, precio, stock y categoría. De manera deliberada, la base contiene errores frecuentes de captura para demostrar la importancia del preprocesamiento de datos: espacios sobrantes, inconsistencias de mayúsculas y minúsculas, registros duplicados, valores faltantes, valores no numéricos y cantidades negativas.

nombre	precio	stock	categoría
Leche Lala 1L	24.5	12	Lacteos
Pan Bimbo Grande	38	8	Panaderia
huevo san juan kg	42	15	Abarrotes
Coca-Cola 600ml	18	30	Bebidas
Leche Lala 1L	24.5	12	Lacteos
Frijol Bayo Kg	32.5	4	Abarrotes
Arroz Morelos Kg	26		Abarrotes
Aceite 123 1L	39.9	3	Abarrotes
Sabritas Original	17.5	25	Botanas
Galletas Marias	14	2	Botanas
AZUCAR ESTANDAR KG	28	10	Abarrotes
Jabon Zote	16.5	6	Limpieza
Papel Higienico 4r	32	5	Limpieza
Atun Dolores	19.5	-3	Abarrotes
Sopa Maruchan	12	40	Abarrotes
cafe nescafe 200g	89	7	Bebidas
Coca-cola 600ML	18	30	Bebidas
Tortillas Maiz Kg	22	18	Abarrotes
Detergente Roma 1kg	29.5		Limpieza
Refresco Sidral 600	17	1	Bebidas
Chiles Jalapenos Lata	15	9	Abarrotes
	25	5	Abarrotes
Mayonesa McCormick	38.5 abc		Abarrotes
Sal La Fina Kg	11	20	Abarrotes

datos >	inventario_muestra.csv
	You, 1 hour ago 1 author (You)
1	nombre,precio,stock,categoria
2	Leche Lala 1L,24.50,12,Lacteos
3	Pan Bimbo Grande,38.00,8,Panaderia
4	huevo san juan kg,42.00,15,Abarrotes
5	Coca-Cola 600ml,18.00,30,Bebidas
6	Leche Lala 1L ,24.50,12,Lacteos
7	Frijol Bayo Kg,32.50,4,Abarrotes
8	Arroz Morelos Kg,26.00,,Abarrotes
9	Aceite 123 1L,39.90,3,Abarrotes
10	Sabritas Original,17.50,25,Botanas
11	Galletas Marias,14.00,2,Botanas
12	AZUCAR ESTANDAR KG,28.00,10,Abarrotes
13	Jabon Zote,16.50,6,Limpieza
14	Papel Higienico 4r,32.00,5,Limpieza
15	Atun Dolores,19.50,-3,Abarrotes
16	Sopa Maruchan,12.00,40,Abarrotes
17	cafe nescafe 200g,89.00,7,Bebidas
18	Coca-cola 600ML,18.00,30,Bebidas
19	Tortillas Maiz Kg,22.00,18,Abarrotes
20	Detergente Roma 1kg,29.50,,Limpieza
21	Refresco Sidral 600,17.00,1,Bebidas
22	Chiles Jalapenos Lata,15.00,9,Abarrotes
23	,25.00,5,Abarrotes
24	Mayonesa McCormick,38.50,abc,Abarrotes
25	Sal La Fina Kg,11.00,20,Abarrotes
26	

Figura 1. contenido del archivo inventario_muestra.csv y captura de VS Code.

3.2 Análisis exploratorio de datos (EDA)

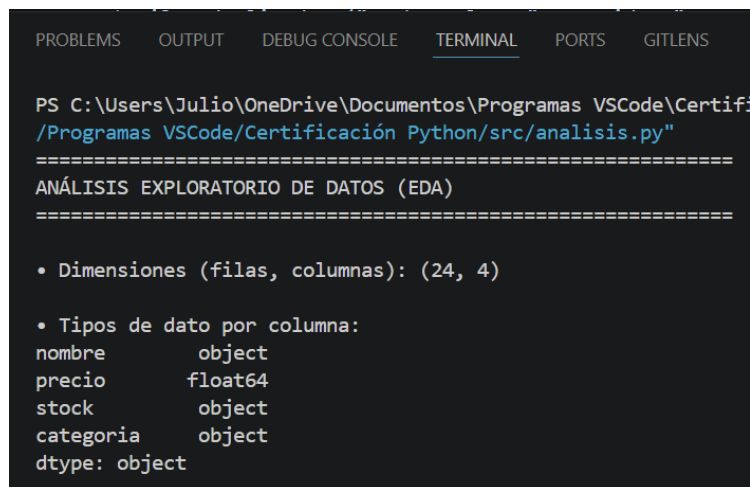
El análisis exploratorio constituye el primer paso para comprender la estructura, la calidad y las posibles anomalías de un conjunto de datos antes de utilizarlo. Mediante la biblioteca pandas se inspeccionaron las dimensiones del conjunto, los tipos de dato de cada columna, los valores faltantes, los registros duplicados y los estadísticos descriptivos de la variable precio (McKinney,

2010). El siguiente fragmento muestra la función de exploración implementada en el módulo `analisis.py`:

```
def explorar(df):
    print(f"Dimensiones: {df.shape}")
    print(df.dtypes)
    print(df.isna().sum())      # valores faltantes
    nombres = df['nombre'].str.strip().str.lower()
    print(nombres.duplicated().sum()) # duplicados
    print(pd.to_numeric(df['precio'], errors='coerce').describe())
```

La ejecución del análisis sobre la base de muestra arrojó los siguientes resultados, que evidencian la presencia de un nombre vacío, dos valores de stock faltantes y dos registros duplicados, además de una columna stock interpretada como texto debido a la presencia de un valor no numérico:

```
Dimensiones (filas, columnas): (24, 4)
Valores faltantes: nombre 1 | precio 0 | stock 2 | categoria 0
Filas duplicadas por nombre (normalizado): 2
Estadísticos de 'precio': media 27.08 | min 11.00 | max 89.00
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\Julio\OneDrive\Documentos\Programas VSCode\Certifi
/Programas VSCode/Certificación Python/src/analisis.py"
=====
ANÁLISIS EXPLORATORIO DE DATOS (EDA)
=====

• Dimensiones (filas, columnas): (24, 4)

• Tipos de dato por columna:
nombre      object
precio      float64
stock       object
categoria   object
dtype: object
```

Figura 2. Salida completa del EDA en la consola.

3.3 Limpieza y preprocesamiento de datos

Tras el diagnóstico, se aplicó un pipeline de limpieza que transforma los datos crudos en un conjunto válido y consistente. El procedimiento normaliza los textos, convierte las columnas numéricas, imputa los valores de stock faltantes con cero, descarta los registros inválidos y elimina los duplicados conservando la primera aparición. Estas operaciones reflejan buenas prácticas de manejo de datos ampliamente recomendadas en la literatura de ciencia de datos.

```
def limpiar(df):
    df['nombre'] = df['nombre'].astype(str).str.strip()
    df['precio'] = pd.to_numeric(df['precio'], errors='coerce')
    df['stock'] = pd.to_numeric(df['stock'], errors='coerce').fillna(0)
    df = df[df['precio'] > 0]      # descarta precios inválidos
    df = df[df['stock'] >= 0]     # descarta stock negativo
```

```
df = df.drop_duplicates(subset='nombre_norm', keep='first')
return df
```

Como resultado del preprocesamiento, el conjunto se redujo de 24 a 20 registros válidos, descartando cuatro filas correspondientes a un registro sin nombre, un valor de stock negativo y dos duplicados. Este resultado confirma que cerca del 17 % de los registros capturados manualmente presentaban algún tipo de inconsistencia, una proporción que ilustra el riesgo de gestionar inventarios sin validaciones automáticas (Ramírez Castillejo & Hernández Pérez, 2023).

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

Limpieza: 24 filas originales -> 20 filas válidas (4 descartadas/duplicadas).

Datos limpios:
      nombre  precio  stock categoria
0  Leche Lala 1L   24.5    12  Lacteos
1  Pan Bimbo Grande  38.0     8 Panaderia
2  huevo san juan kg  42.0    15  Abarrotes
3  Coca-Cola 600ml  18.0    30  Bebidas
4  Frijol Bayo Kg   32.5     4  Abarrotes
5  Arroz Morelos Kg  26.0     0  Abarrotes
6  Aceite 123 1L   39.9     3  Abarrotes
7  Sabritas Original 17.5    25  Botanas
8  Galletas Marias  14.0     2  Botanas
9  AZUCAR ESTANDAR KG 28.0    10  Abarrotes
10 Jabon Zote      16.5     6  Limpieza
11 Papel Higienico 4r 32.0     5  Limpieza
12 Sopa Maruchan   12.0    40  Abarrotes
13 cafe nescafe 200g 89.0     7  Bebidas
14 Tortillas Maiz Kg 22.0    18  Abarrotes
15 Detergente Roma 1kg 29.5     0  Limpieza
16 Refresco Sidral 600 17.0     1  Bebidas
17 Chiles Jalapenos Lata 15.0     9  Abarrotes
18 Mayonesa McCormick 38.5     0  Abarrotes
19 Sal La Fina Kg   11.0    20  Abarrotes
```

Figura 3. Datos limpios impresa en la consola.

4. Evaluación del programa: funciones y técnicas utilizadas

Esta sección describe las principales técnicas de programación empleadas, su función dentro del sistema y sus restricciones. El proyecto se estructuró en módulos independientes (modelo, inventario, análisis e interfaz) siguiendo el principio de separación de responsabilidades, lo que facilita el mantenimiento y la legibilidad del código (Van Rossum & Drake, 2009).

4.1 Programación orientada a objetos (POO)

El sistema se modeló mediante dos clases principales. La clase Producto representa la unidad básica de información y encapsula sus atributos (nombre, precio, stock y categoría) junto con la validación de las reglas de negocio en su constructor. La clase Inventario actúa como contenedor y orquesta todas las operaciones sobre la colección de productos. El uso de POO permite reutilizar código y mantener la integridad del estado de los objetos (Van Rossum & Drake, 2009).

```

@dataclass
class Producto:
    nombre: str
    precio: float
    stock: int
    categoria: str = 'General'
    def __post_init__(self):
        if self.precio <= 0:
            raise ProductoInvalidoError('precio debe ser > 0')

```

Se empleó el decorador `dataclass` para reducir el código repetitivo de los métodos especiales y la propiedad `property` para exponer valores derivados, como el valor monetario del inventario. Como restricción, la validación se concentra en la creación del objeto; modificaciones posteriores directas a los atributos no se revalidan automáticamente, por lo que las operaciones se canalizan a través de métodos de la clase `Inventario` (Van Rossum & Drake, 2009).

4.2 Estructuras de datos avanzadas

El control de productos se apoya en un diccionario (`dict`) que asocia una clave normalizada con cada objeto `Producto`, lo que permite acceder, actualizar o eliminar productos con una complejidad temporal constante, es decir $O(1)$ en promedio. De forma complementaria, un conjunto (`set`) de claves se utiliza para verificar de manera inmediata la existencia de un producto y así impedir duplicados. Las listas se reservan para devolver resultados ordenados destinados a su visualización (Van Rossum & Drake, 2009).

La principal restricción de estas estructuras es que residen en memoria volátil: su contenido se pierde al cerrar el programa. Por ello, el sistema las complementa con un mecanismo de persistencia en disco que se describe en la sección 5 (McKinney, 2010).

4.3 Manipulación de datos con pandas y persistencia con JSON

La biblioteca `pandas` se utilizó para el análisis exploratorio y la limpieza de la base de datos de muestra, aprovechando su estructura `DataFrame` para operar sobre columnas completas de forma vectorizada y eficiente (McKinney, 2010). Para la persistencia se eligió el formato JSON por ser un estándar ligero, legible por humanos y nativo de Python a través del módulo `json`, lo que facilita guardar y recuperar el inventario entre sesiones sin necesidad de un sistema gestor de bases de datos (Van Rossum & Drake, 2009).

4.4 Visualización con matplotlib

Para el análisis estadístico visual se empleó la biblioteca `matplotlib`, estándar de facto para la generación de gráficos en el ecosistema científico de Python (Hunter, 2007). Se generaron tres visualizaciones: un gráfico de barras de existencias por producto con resaltado del bajo stock, un gráfico de pastel de la distribución del valor por categoría y un histograma de la distribución de precios. Como restricción, las gráficas se generan con un backend no interactivo que las exporta como imágenes, decisión que favorece la automatización, pero omite la interactividad en pantalla (Hunter, 2007).

5. Complementación: control de datos y persistencia

5.1 Control de datos: duplicados y bajo stock

El control de datos garantiza la calidad de la información del inventario. Para evitar productos duplicados, antes de dar de alta un producto el sistema verifica si su clave normalizada ya existe en el conjunto de claves; en caso afirmativo, rechaza la operación y sugiere utilizar la función de reabastecimiento. Este mecanismo previene la fragmentación de la información, un problema común en registros manuales (Pin Figueroa et al., 2019).

```
def agregar_producto(self, producto):
    if producto.clave in self._claves:    # control de duplicados
        raise ProductoInvalidoError('el producto ya existe')
    self._productos[producto.clave] = producto
    self._claves.add(producto.clave)
```

Asimismo, el sistema identifica los productos cuyo nivel de existencias es igual o inferior a un umbral configurable (cinco unidades por defecto), generando una alerta de bajo stock. Esta funcionalidad permite al comerciante anticipar reabastecimientos y evitar desabastos que afectan las ventas (Pin Figueroa et al., 2019).

5.2 Persistencia de datos

La persistencia asegura que el inventario permanezca disponible entre ejecuciones del programa. Cada operación que modifica el estado del inventario desencadena un guardado automático en el archivo inventario.json, y al iniciar el programa este se carga de forma automática. El proceso de carga incorpora manejo de excepciones para que un archivo inexistente o dañado no interrumpa la ejecución, iniciando en tal caso con un inventario vacío (Van Rossum & Drake, 2009).

```
def guardar(self):
    datos = [p.a_dict() for p in self.listar()]
    with open(self.ruta_archivo, 'w', encoding='utf-8') as f:
        json.dump(datos, f, ensure_ascii=False, indent=4)
```

Esta estrategia de persistencia ligera resulta adecuada para el volumen de datos de una tienda pequeña y elimina la dependencia de infraestructura adicional, alineándose con la necesidad de soluciones accesibles para las microempresas (Romero-Agila et al., 2021).

5.3 Análisis estadístico visual

Como complemento analítico, el sistema genera visualizaciones que resumen el estado del inventario y apoyan la toma de decisiones. La Figura 4 muestra las existencias por producto, resaltando en color los artículos por debajo del umbral de bajo stock (Hunter, 2007).

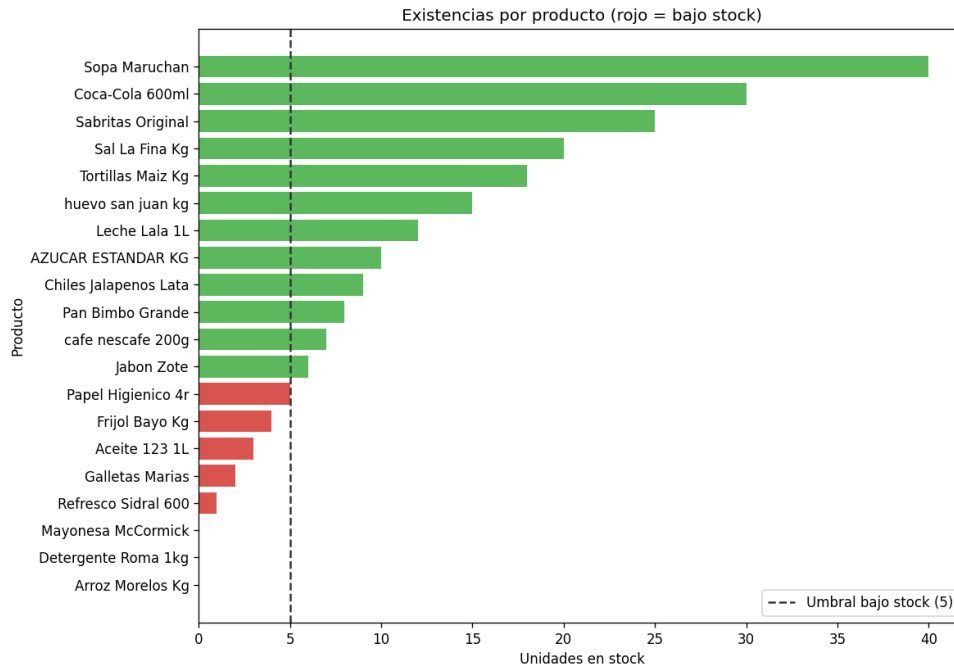


Figura 4. Existencias por producto con resultado de bajo stock.

La Figura 4 presenta la distribución del valor monetario del inventario por categoría, información útil para identificar en qué tipo de productos se concentra el capital invertido del negocio (Hunter, 2007).

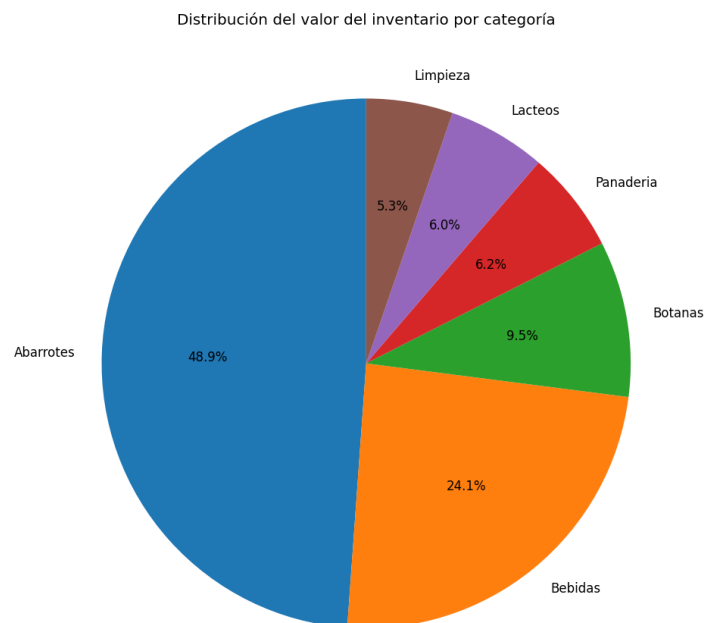


Figura 5. Distribución del valor del inventario por categoría.

6. Instrucciones de uso paso a paso

A continuación, se describe el procedimiento para ejecutar e interactuar con el sistema. Se incluyen los comandos, una descripción del resultado esperado y espacios reservados para las capturas de pantalla correspondientes.

Paso 1. Instalar las dependencias

Desde la carpeta del proyecto, ejecutar el siguiente comando para instalar las bibliotecas necesarias (pandas, matplotlib y python-docx):

```
pip install -r requirements.txt
```

Paso 2. Ejecutar el programa

Iniciar la aplicación con el intérprete de Python. El sistema cargará automáticamente el inventario guardado y mostrará el menú principal:

```
python src/main.py
```

```
=====
      SISTEMA DE GESTIÓN DE INVENTARIOS - Tienda de Abarrotes
=====
1. Mostrar inventario
2. Agregar producto
3. Reabastecer producto
4. Registrar venta (descontar stock)
5. Eliminar producto
6. Ver productos con bajo stock
7. Cargar base de datos de muestra (EDA + limpieza)
8. Generar gráficas estadísticas
0. Salir
=====
Elige una opción: |
```

Figura 6. menú principal del sistema.

Paso 3. Mostrar el inventario

Seleccionar la opción 1 para visualizar la tabla completa de productos, con su precio, stock, categoría y el valor total del inventario:

```
Elige una opción: 1
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

=====
Elige una opción: 1

PRODUCTO | PRECIO | STOCK | CATEGORÍA
-----|-----|-----|-----
Aceite 123 1L | $ 39.90 | 3 u | Abarrotes
Arroz Morelos Kg | $ 26.00 | 0 u | Abarrotes
AZUCAR ESTANDAR KG | $ 28.00 | 10 u | Abarrotes
cafe nescafe 200g | $ 89.00 | 7 u | Bebidas
Chiles Jalapenos Lata | $ 15.00 | 9 u | Abarrotes
Coca-Cola 600ml | $ 18.00 | 30 u | Bebidas
Detergente Roma 1kg | $ 29.50 | 0 u | Limpieza
Frijol Bayo Kg | $ 32.50 | 4 u | Abarrotes
Galletas Marias | $ 14.00 | 2 u | Botanas
huevo san juan kg | $ 42.00 | 15 u | Abarrotes
Jabon Zote | $ 16.50 | 6 u | Limpieza
Leche Lala 1L | $ 24.50 | 12 u | Lacteos
Mayonesa McCormick | $ 38.50 | 0 u | Abarrotes
Pan Bimbo Grande | $ 38.00 | 8 u | Panaderia
Papel Higienico 4r | $ 32.00 | 5 u | Limpieza
Refresco Sidral 600 | $ 17.00 | 1 u | Bebidas
Sabritas Original | $ 17.50 | 25 u | Botanas
Sal La Fina Kg | $ 11.00 | 20 u | Abarrotes
Sopa Maruchan | $ 12.00 | 40 u | Abarrotes
Tortillas Maiz Kg | $ 22.00 | 18 u | Abarrotes
-----
Valor total del inventario: $4,893.20 MXN

Presiona ENTER para continuar...|
```

Figura 7. Inventario Principal de la tienda de abarrotes.

Paso 4. Agregar un producto

Seleccionar la opción 2 e ingresar los datos solicitados. Si el producto ya existe, el sistema lo notificará para evitar duplicados:

Elige una opción: 2
Nombre del producto: Chocolate Abuelita
Precio (MXN): 22.50
Stock inicial (unidades): 10
Categoría: Bebidas

```
=====
Elige una opción: 2
Nombre del producto: Chocolate Abuelita
Precio (MXN): 22.50
Stock inicial (unidades): 10
Categoría (ENTER = General): Bebidas
✓ Producto 'Chocolate Abuelita' agregado y guardado.

Presiona ENTER para continuar...
```

Figura 8. Alta de un producto nuevo.

Paso 5. Reabastecer y registrar ventas

Las opciones 3 y 4 permiten sumar o descontar unidades del stock. El sistema valida que exista suficiente inventario antes de registrar una venta:

Elige una opción: 3
Producto a reabastecer: Frijol Bayo Kg
Unidades a agregar: 20

```
=====
Elige una opción: 3
Producto a reabastecer: Frijol Bayo Kg
Unidades a agregar: 20
  ✓ 'Frijol Bayo Kg' ahora tiene 24 unidades.

Presiona ENTER para continuar...
```

Figura 9. Reabastecimiento de un producto.

Paso 6. Consultar bajo stock

La opción 6 lista los productos cuyo nivel de existencias requiere atención, facilitando la planeación de compras:

Elige una opción: 6

```
=====
Elige una opción: 6

△Productos con bajo stock (<= 5 unidades):
-----
Arroz Morelos Kg      | $ 26.00 | 0 u | Abarrotes
Detergente Roma 1kg   | $ 29.50 | 0 u | Limpieza
Mayonesa McCormick    | $ 38.50 | 0 u | Abarrotes
Refresco Sidral 600   | $ 17.00 | 1 u | Bebidas
Galletas Marias       | $ 14.00 | 2 u | Botanas
Aceite 123 1L         | $ 39.90 | 3 u | Abarrotes
Papel Higienico 4r     | $ 32.00 | 5 u | Limpieza

Presiona ENTER para continuar...
```

Figura 10. Reporte de bajo stock.

Paso 7. Cargar la base de muestra y generar gráficas

Las opciones 7 y 8 permiten cargar la base de datos de ejemplo (ejecutando el EDA y la limpieza) y generar las gráficas estadísticas en la carpeta graficas:

Elige una opción: 7 y después elige la opción 8.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

• Valores faltantes por columna:
nombre      1
precio      0
stock       2
categoria   0
dtype: int64

• Filas duplicadas por nombre (normalizado): 2

• Estadísticos de 'precio':
count      24.000000
mean       27.079167
std        16.024790
min        11.000000
25%        17.375000
50%        24.500000
75%        32.125000
max        89.000000
Name: precio, dtype: float64
=====

Limpieza: 24 filas originales -> 20 filas válidas (4 descartadas/duplicadas).

✓ 0 productos nuevos cargados desde la muestra y guardados.

Presiona ENTER para continuar...

=====
      SISTEMA DE GESTIÓN DE INVENTARIOS - Tienda de Abarrotes
=====

1. Mostrar inventario
2. Agregar producto
3. Reabastecer producto
4. Registrar venta (descontar stock)
5. Eliminar producto
6. Ver productos con bajo stock
7. Cargar base de datos de muestra (EDA + limpieza)
8. Generar gráficas estadísticas
0. Salir

=====
Elige una opción: 8

✓ Gráficas generadas en la carpeta 'graficas/':
- graficas\stock_por_producto.png
- graficas\valor_por_categoria.png
- graficas\distribucion_precios.png

Presiona ENTER para continuar...
```

Figura 11. EDA y gráficas generadas.

7. Conclusión

El desarrollo de este sistema de gestión de inventarios permitió integrar y aplicar de manera práctica los temas revisados durante la certificación en Python: desde los fundamentos del

lenguaje y las estructuras de control, hasta la programación orientada a objetos, las estructuras de datos avanzadas, la manipulación de datos con pandas y la visualización con matplotlib. La construcción modular del proyecto evidenció la importancia de las buenas prácticas de codificación, como la separación de responsabilidades, la documentación mediante docstrings y el manejo controlado de excepciones.

Más allá del ejercicio técnico, el proyecto reafirma el valor de la programación como herramienta para resolver problemas reales. Python, por su accesibilidad y su robusto ecosistema de bibliotecas, demuestra ser un lenguaje idóneo para construir soluciones que, sin requerir infraestructura costosa, aportan valor tangible a contextos cotidianos. La capacidad de transformar datos crudos y desordenados en información confiable es, precisamente, una de las competencias más demandadas en la actualidad.

En el ámbito de las microempresas, sistematizar la gestión de inventarios trasciende la mera digitalización: incide en la rentabilidad, reduce pérdidas y fortalece la toma de decisiones, factores asociados con una mayor probabilidad de supervivencia empresarial en un entorno donde la mortalidad de los negocios es elevada. Por ello, soluciones accesibles como la aquí presentada tienen el potencial de contribuir, aunque sea de forma modesta, a la profesionalización y permanencia de las tiendas de abarrotes mexicanas.

8. Referencias

- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Instituto Nacional de Estadística y Geografía. (2024a). Estadísticas a propósito del Día de las Micro, Pequeñas y Medianas Empresas (Comunicado de prensa núm. 383/24). INEGI. https://www.inegi.org.mx/contenidos/saladeprensa/aproposito/2024/EAP_MIPYMES24.pdf
- Instituto Nacional de Estadística y Geografía. (2024b). Estudio sobre la Demografía de los Negocios (EDN) 2023 (Comunicado de prensa núm. 68/24). INEGI. <https://www.inegi.org.mx/contenidos/saladeprensa/boletines/2024/EDN/EDN2023.pdf>
- McKinney, W. (2010). Data structures for statistical computing in Python. En *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- Pin Figueroa, F. E., Pin Figueroa, M. A., & Pin Figueroa, J. L. (2019). El control de los inventarios y su incidencia en las decisiones gerenciales en las microempresas de comercio de Jipijapa. *FIPCAEC*, 4(1 Especial), 1–20. <https://www.fipcaec.com/index.php/fipcaec/article/view/106>
- Ramírez Castillejo, M. de los Á., & Hernández Pérez, J. (2023). La ausencia de la gestión administrativa en las pymes de México, una realidad que les impide crecer. *Revista IPSUMTEC*, 6(2). <https://revistas.milpaalta.tecnm.mx/index.php/IPSUMTEC/article/view/137>
- Romero-Agila, M. L., Erazo-Álvarez, J. C., Narváez-Zurita, C. I., & Matovelle-Romo, M. (2021). La gestión de inventarios en las PYMES del sector de la construcción. *Polo del Conocimiento*, 6(9), 100–124. <https://doi.org/10.23857/pc.v6i9.3124>
- Van Rossum, G., & Drake, F. L. (2009). *The Python language reference manual*. Python Software Foundation. <https://docs.python.org/3/reference/>